

TP4 – Pilotage de LED et mémoire

Partie 2

Christophe Dufaza

Table des matières

Question 1	2
Architecture RTL	2
Interface	2
Circuit LED <i>driver</i>	2
Question 2	3
FIFO Generator	3
Implémentation	3
Reset et initialisation	4
Configuration	4
Interface	5
Architecture RTL	5
Interface	5
Circuit <i>LED Jukebox</i>	6
Question 3	6
Architecture VHDL	6
Question 4	7
Simulations	7
Initialisation, écriture, lecture	7
Dépassement de capacité (FIFO <i>full</i>)	8
Question 5	9
Synthèse	9
Rapports d'utilisation	9
Ressources FIFO	9
Question 6	11
Implémentation	11
Place & route	11
Rapports de <i>timing</i>	11
Question 7	12
Démonstration	12

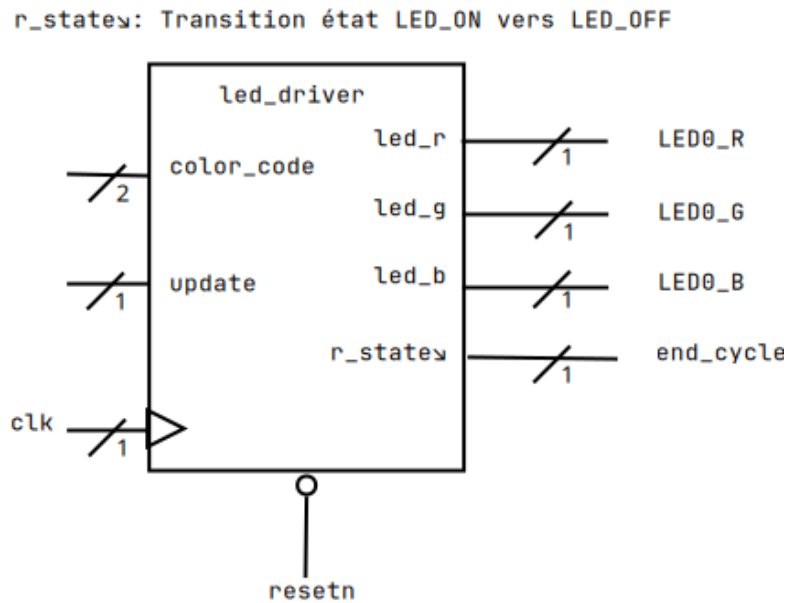


Figure 1: Circuit LED *driver* Question 1.

Question 1

Sur l'architecture RTL, modifiez le module `LED_driver` en ajoutant une sortie `end_cycle`. Cette sortie vaudra 1 à la fin d'un cycle allumé/éteint de la LED RGB.

Architecture RTL

Interface

Les ports I/O du circuit `led_driver` ainsi modifié sont rappelés Table 1.

Table 1: Ports I/O LED driver Question 1.

Port	Type	Signal
<code>clk</code>	I	Horloge
<code>resetn</code>	I	RST asynchrone, active-low
<code>update</code>	I	Enregistrement (synchrone) code couleur <i>enabled</i>
<code>color_code[1:0]</code>	I	Code couleur
<code>led_r</code>	O	Signal pilote (<i>driver</i>) LED rouge
<code>led_g</code>	O	Signal pilote (<i>driver</i>) LED verte
<code>led_b</code>	O	Signal pilote (<i>driver</i>) LED bleue
<code>end_cycle</code>	O	<i>Pulse</i> de fin de cycle LED_ON / LED_OFF

Le circuit est paramétré par un diviseur de fréquence D .

Table 2: Paramètre LED driver Question 1.

Générique	Configuration
D	Diviseur de fréquence

Circuit LED *driver*

Le signal de sortie `end_cycle` est actif, pendant une période, lors des transitions de l'état `LED_ON` vers `LED_OFF`. Le circuit est décrit Fig. 1.

Question 2

Modifiez la logique en entrée du module pour ajouter une FIFO. Cette FIFO doit prendre en entrée le code couleur « vert » ou « bleu » suivant l'état du `bouton_1` et est connectée en sortie à l'entrée `color_code` du module `LED_driver`. La donnée est écrite dans la FIFO lorsqu'il y a un front montant du `bouton_0`. La donnée de la FIFO est lue lorsque le signal `end_cycle` du module `LED_driver` vaut 1.

FIFO Generator

Les mémoires FIFO proposées par le catalogue IP Vivado couvrent une multitude d'options (notamment implémentation, profondeur, largeur, horloge et *reset*) et d'interfaces de communication (native, AXI4, AXI4-Stream, AXI4-Lite).

Quelles sont nos spécifications:

- Horloge commune aux domaines lecture et écriture (*Common Clock*).
- *Reset* asynchrone.
- Interface native.
- Largeur (*width*) pour codes couleur 2-bit.
- Profondeur (*depth*) suffisante pour les codes couleur qu'un utilisateur peut enregistrer, en deux secondes ¹, en manipulant les boutons de contrôle.

Le composant **FIFO Generator** (13.2) supporte l'interface native et peut être configuré pour une interface I/O minimale: statuts essentiels (`full`, `empty`), écriture (`wr_en`, `din`) et lecture (`rd_en`, `dout`). Ce sera notre choix pour ce circuit.

Le manuel de référence pour cet IP est PG057.

Implémentation

L'implémentation built-in est écartée:

- Spécifique à une famille de cartes, et donc moins portable.
- Surdimensionnée, par exemple la profondeur minimale est 512 lectures/écritures.
- Initialisation inutilement complexe ici, par exemple les signaux `empty` et `full` sont tous deux désactivés pendant un *reset*, d'autres signaux sont nécessaires pour observer l'initialisation de la mémoire FIFO.

Pour de petites FIFO (largeur $\times$ profondeur), les implémentations Distributed RAM (LUTs) et Shift Register sont souvent préférées à des blocs RAM dédiés BRAM que l'on voudra réserver pour de réels besoins de mémorisation ².

Par ailleurs, les transitions d'état `LED_ON` vers `LED_OFF`, après optimisation et routage, sont implémentées par un circuit purement combinatoire dont certaines entrées sont connectées au registre du compteur *interne* `counter_unit`. Et ainsi, *in fine*, le signal d'entrée FIFO `rd_en` est piloté par l'état d'un registre admettant un *reset* asynchrone.

A moins de mettre en oeuvre le protocole *Safety Circuit*, une implémentation FIFO BRAM, lors du placement et routage, produira de nombreux avertissements comme ci-dessous:

```
[DRC REQ-1840] The RAMB18E1 has an input control pin ... which is
driven by a register ... that has an active asynchronous set or reset.
```

Ces avertissements peuvent ne pas paraître critiques ³, mais mettent néanmoins en évidence un *hiatus* de conception.

L'implémentation **Shit Register**, adaptée aux petites mémorisations et *reset* asynchrone sera choisie ici.

¹Période de lecture FIFO.

²Il s'agit ici de mémoriser quelques mots de 2 bits.

³Le comportement du circuit avec cette implémentation FIFO semble malgré tout correct en simulation.

Reset et initialisation

Avant toute écriture ou lecture, l'implémentation FIFO doit être initialisée tel que décrit PG057 §3 Asynchronous Reset, notamment:

IMPORTANT: The clock(s) must be available when the reset is applied.

Note: It is always recommended to have the asynchronous reset asserted for at least 3 or C_SYNCHRONIZER_STAGE (whichever is maximum) slowest clock cycles.

Full Flags Reset Value of 1

In this configuration, the FIFO requires a minimum asynchronous reset pulse of 1 write/read clock cycle (whichever is slower). After reset is detected on the rising clock edge of write clock, 3 write clock periods are required to complete proper reset synchronization. During this time, the **full** flag is asserted. After reset is deasserted, this flag deasserts after five clock periods and the FIFO can then accept write operations.

Table 3: Asynchronous Reset Values for Block, Distributed, and Shift RAM FIFOs

Signal	Full Flags Reset Value of 1	Power-up Value
dout	dout Reset Value or 0	Same as reset values
full	1	0
empty	1	1

Nous retiendrons:

- Une pulsation *reset* d'au moins 3 cycles d'horloge est recommandée ⁴.
- L'initialisation est complète 5 cycles d'horloge après dé-assertion du signal **rst**.
- Pendant l'ensemble de l'initialisation, les deux signaux **full** et **empty** sont actifs.

Configuration

La configuration adoptée ici est résumée Table 4.

Table 4: Configuration FIFO Generator 13.

Interface Type	Native
Fifo Implementation	Common Clock Shift Register
Read Mode	Standard FIFO ⁵
Write Width	2
Write Depth	16
Read Width	2
Read Depth	16
Reset Pin	Asynchronous Reset
Full Flags Reset Value	1
Dout Reset Value	0

⁴On trouve dans les sources IP VHDL C_SYNCHRONIZER_STAGE = 2.

⁵La configuration First Word Fall Through (FWTH), également évaluée, ne semble ni utile ni nécessaire à notre scénario d'utilisation.

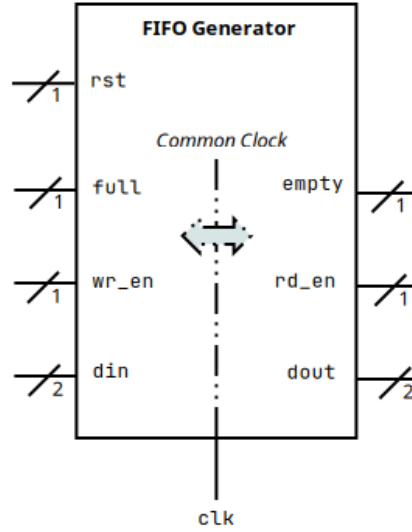


Figure 2: Interface FIFO Generator.

Interface

L'interface de l'IP FIFO Generator ainsi configuré est donnée Fig 2.

Architecture RTL

Il s'agit donc de décrire une architecture RTL programmant les codes couleurs d'un `led_driver` via une mémoire FIFO Generator.

Ce circuit sera nommé `led_jukebox`.

Interface

Les ports I/O sont donnés Table 5.

Table 5: Ports I/O *LED Junk Box*.

Port	Type	Signal
<code>clk</code>	I	Horloge
<code>resetn</code>	I	Reset asynchrone, active-low
<code>BTN0</code>	I	Signal requête enregistrement code couleur
<code>BTN1</code>	I	Signal sélection code couleur
<code>LED0_R</code>	O	Signal pilote (<i>driver</i>) LED rouge
<code>LED0_G</code>	O	Signal pilote (<i>driver</i>) LED verte
<code>LED0_B</code>	O	Signal pilote (<i>driver</i>) LED bleue

Le circuit est paramétré par un diviseur de fréquence D tel que $T_{led} = D \times T_{clk}$.

Par exemple, si l'on souhaite un clignotement de période deux secondes (LED allumée une seconde, éteinte une seconde), à partir d'une horloge 100 MHz, on paramétrera le circuit avec $D = \frac{2}{10^{-8}} = 2 \cdot 10^8$.

Le circuit est également paramétré par la durée d'une impulsion *reset*, exprimée en cycles d'horloge.

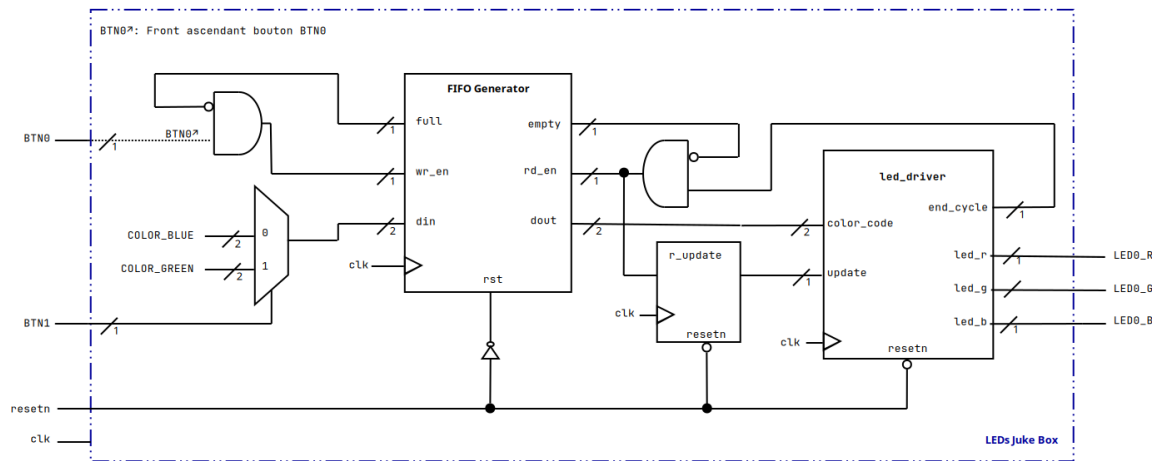


Figure 3: Circuit *LEDs Jukebox*.

Table 6: Paramètres *LED Junk Box*.

Générique	Configuration
D	Diviseur de fréquence
RST_SYNC_STAGE	Durée d'une impulsion <i>reset</i> (cycles d'horloge)

Circuit *LED Jukebox*

Le circuit *LED Jukebox* est décrit Fig. 3.

Le registre `r_update` synchronise le signal `update` en entrée du circuit `led_driver` avec la disponibilité des données (le nouveau code couleur) sur le port de sortie `dout` de la mémoire FIFO.

La logique combinatoire en entrée des signaux `rd_en` et `wr_en` permet d'éviter toute lecture d'une mémoire vide, ou écriture dans une mémoire pleine.

Question 3

Modifiez vos codes de la partie 1 pour y ajouter les nouveaux éléments de votre architecture.

Architecture VHDL

Description VHDL du circuit Question 2: `src/design/led_jukebox.vhd`

La description VHDL reprend littéralement la schématique Fig. 3, à l'exception du mécanisme d'initialisation qui n'y est pas représenté:

- Bien qu'une période d'horloge soit *a priori* suffisante, il est recommandé que le signal `rst` soit actif pendant trois périodes: la durée de l'initialisation de la mémoire FIFO, exprimée en cycles d'horloge, sera paramétrée par le *generic* `RST_SYNC_STAGE`.
- Lors du démarrage⁶ du circuit, le registre `r_fifo_sync_stage` est initialisé avec le paramètre `RST_SYNC_STAGE`, puis décrémenté à chaque front d'horloge montant: le signal `rst` sera alors actif tant que la valeur enregistrée dans `r_fifo_sync_stage` sera différente de zéro.

⁶Ou d'un *reset*.

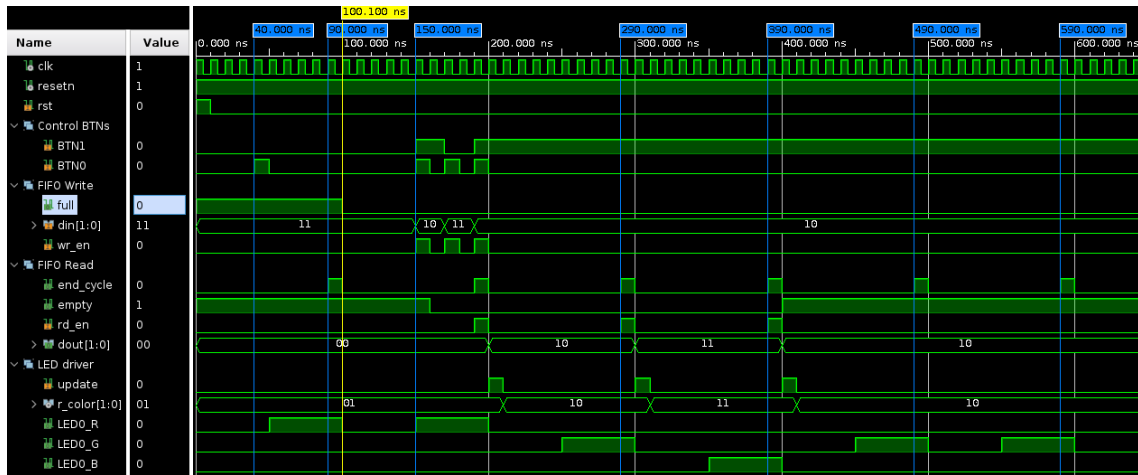


Figure 4: Chronographe *testbench tb_led_jukebox*.

Question 4

Mettez à jour le testbench et réalisez une simulation pour vérifier votre design.

Simulations

Initialisation, écriture, lecture

- *Testbench* VHDL: `src/sim/tb_led_jukebox.vhd`,
- Configuration *waveform*: `src/sim/tb_led_jukebox.wfg`

Pour cette simulation:

- Le *reset* est configuré pour une durée d'un cycle d'horloge: `RST_SYN_STAGE = 1`
- Le diviseur de fréquence est configuré afin qu'une période `end_cycle` corresponde à 10 périodes d'horloge: `D = 10`

Le comportement simulé du circuit est vérifié par les tests automatiques (`assert`), et illustré par le chronographe Fig. 4:

- Au démarrage (*power-on*) le signal d'entrée `rst` est actif, l'initialisation de la mémoire FIFO débute: pendant cette initialisation les *flags* `empty` et `full` sont tous deux levés.
- À $t = 40$ ns, un *click* sur le bouton de contrôle `BTN0` est simulé: le signal `full` étant activé, l'opération d'écriture en mémoire FIFO n'est pas initiée.
- À $t = 90$ ns, première transition de l'état `LED_ON` vers `LED_OFF` (signal `end_cycle`): le signal `empty` étant activé, l'opération de lecture de la mémoire FIFO n'est pas initiée.
- À $t = 100$ ns, l'initialisation de la mémoire FIFO est terminée, le signal `full` est désactivé, les opérations d'écriture peuvent débuter dès le cycle d'horloge suivant.
- À partir de $t = 150$ ns, sont simulés trois *clicks* sur le bouton de contrôle `BTN0`, le bouton `BTN1` spécifiant successivement les couleur vert, bleu, vert: on vérifie les opération d'écriture correspondantes sur les signaux d'entrée du FIFO `wr_en` et `din`.
- À $t = 190$ ns, deuxième transition de l'état `LED_ON` vers `LED_OFF`: l'opération de lecture correspondante est initiée en activant le signal `rd_en` en entrée du FIFO.
- À $t = 200$ ns, c'est à dire lors du cycle d'horloge suivant, le signal `update` notifie au composant `led_driver` qu'un nouveau code couleur est disponible sur le port de sortie `dout` du FIFO.
- À $t = 210$ ns, soit deux cycles d'horloge après l'impulsion `end_cycle`, la couleur vert est observable dans le registre `r_color` du *driver* LED.
- Deux nouvelles opérations de lecture sont initiées lors des transitions suivantes de l'état `LED_ON` vers `LED_OFF`, à $t = 290$ ns puis $t = 390$ ns.
- Lorsque le troisième code couleur est lu dans la mémoire FIFO, celle-ci est vide, et le *flag* `empty` est activé ($t = 400$ ns).
- Lors des impulsion `end_cycle` suivantes, à $t = 490$ ns puis $t = 590$ ns, aucune opération de lecture ne sera initiée, et la couleur ne changera plus.

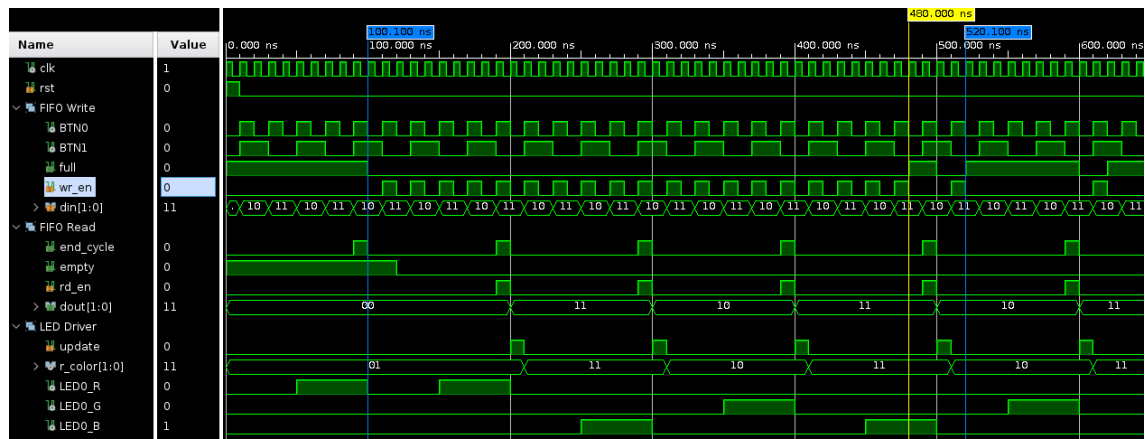


Figure 5: Chronographe *testbench* `tb_led_jukebox_full`.

Dépassement de capacité (FIFO *full*)

- *Testbench* VHDL: `src/sim/tb_led_jukebox_full.vhd`,
- Configuration *waveform*: `src/sim/tb_led_jukebox_full.wfg`

Pour cette simulation:

- Le *reset* est configuré pour une durée d'un cycle d'horloge: `RST_SYN_STAGE = 1`
- Le diviseur de fréquence est configuré afin qu'une période `end_cycle` corresponde à 10 périodes d'horloge: `D = 10`

Le *testbench* injecte continuellement, via les boutons de contrôle, les couleurs vert et bleu dans le module LED driver, comme illustré Fig. 5:

- De même que précédemment, lecture et écriture dans la mémoire FIFO sont suspendus pendant son initialisation.
- Puis, pour chaque période du signal `end_cycle`, on observe une lecture FIFO pour 5 écritures, jusqu'à exhaustion de la mémoire.
- À partir de quoi, chaque période du signal `end_cycle` ne comprendra qu'une lecture et une écriture FIFO.

Question 5

Réalisez une synthèse et étudiez le rapport de synthèse, les ressources utilisées doivent correspondre à votre schéma RTL.

Synthèse

Rapports d'utilisation

- Utilization Design Information: `synth/led_jukebox_utilization_synth.rpt`
- Synthesis report: `synth/led_jukebox.vds`

Table 7: Summary of Registers by Type.

Total	Clock Enable	Synchronous	Asynchronous
3	Yes	-	Set
31	Yes	-	Reset

Table 8: Primitives (synthèse).

Ref Name	Used	Functional Category
FDCE	31	Flop & Latch
LUT4	21	LUT
LUT3	8	LUT
CARRY4	7	CarryLogic
LUT5	5	LUT
LUT6	4	LUT
IBUF	4	IO
OBUF	3	IO
LUT2	3	LUT
LUT1	3	LUT
FDPE	3	Flop & Latch
BUFG	1	Clock

Comparé aux ressources utilisées par le circuit `top_led_driver` de la Partie 1, nous observons:

- Deux nouveaux registres FDPE (D Flip-Flop with Clock Enable and Asynchronous Preset), synthétisant le compteur descendant 2-bit `r_fifo_sync_stage`.
- Un registre FDCE supplémentaire, synthétisant le registre de synchronisation `r_update`.

Le circuit comprend également davantage de LUT afin d'implémenter la logique combinatoire supplémentaire, notamment pour piloter les signaux `rd_en` et `wr_en` en entrée de la mémoire FIFO.

Les ressources utilisées par la mémoire FIFO, *black box* `fifo_gen_shiftreg`, sont données dans des rapports d'utilisation différents.

Ressources FIFO

- Utilization Design Information: `synth/fifo_gen_shiftreg_utilization_synth.rpt`
- Synthesis report: `synth/fifo_gen_shiftreg.vds`

Table 9: Summary of Registers by Type.

Total	Clock Enable	Synchronous	Asynchronous
15	Yes	-	Set
9	Yes	-	Reset
4	Yes	Reset	-

Table 10: Primitives (synthèse).

Ref Name	Used	Functional Category
LUT1	17	LUT
FDPE	15	Flop & Latch
FDCE	9	Flop & Latch
LUT4	8	LUT
LUT6	7	LUT
LUT5	4	LUT
FDRE	4	Flop & Latch
LUT2	3	LUT
SRLC16E	2	Distributed Memory
LUT3	1	LUT

Les éléments de mémorisation sont les deux Shift Registers **SRLC16E** (2 mots 16-bit), les autres éléments représentent essentiellement le protocole I/O, le bloc *reset* **rstblk**, et les registres *pipeline* en sortie mémoire.

Question 6

Effectuez le placement routage et étudiez les rapports.

Implémentation

À partir de l'*implémentation*, nous utiliserons le fichier de contraintes `src/const/led_jukebox.xdc`.

Place & route

- IO Place Design: `impl/led_jukebox_io_placed.rpt`
- Utilization - Place Design: `impl/led_jukebox_utilization_placed.rpt`

Les ressources utilisées après placement et routage correspondent aux ressources du circuit `led_jukebox` plus les ressources internes à la mémoire FIFO `fifo_gen_shiftreg`.

Table 11: Summary of Registers by Type.

Total	Clock Enable	Synchronous	Asynchronous
17	Yes	-	Set
40	Yes	-	Reset
4	Yes	Reset	-

Table 12: Primitives (implémentation).

Ref Name	Used	Functional Category
FDCE	40	Flop & Latch
LUT4	29	LUT
FDPE	17	Flop & Latch
LUT1	14	LUT
LUT6	11	LUT
LUT5	9	LUT
LUT3	9	LUT
CARRY4	7	CarryLogic
LUT2	6	LUT
IBUF	4	IO
FDRE	4	Flop & Latch
OBUF	3	IO
SRL16E	2	Distributed Memory
BUFG	1	Clock

Rapports de *timing*

- Timing Summary - Placed Design: `impl/led_jukebox_timing_summary_placed.rpt`
- Timing Summary - Route Design: `impl/led_jukebox_timing_summary_routed.rpt`
- Clock Utilization - Route Design: `impl/led_jukebox_clock_utilization_routed.rpt`

On vérifie dans le rapport de *timing* que l'analyse du circuit *implémenté* produit de même des valeurs nulles pour *Total Negative Slack* (TNS) et *Total Hold Slack* (THS).

Table 13: Design Timing Summary.

WNS(ns)	TNS(ns)	WHS(ns)	THS(ns)	WPWS(ns)	TPWS(ns)
4.354	0.000	0.122	0.000	4.020	0.000

L'horloge pilotant le circuit confirme la configuration décrite dans le fichier de contraintes (fréquence, *duty cycle*, phase ⁷).

⁷Délai initial avant le premier front d'horloge montant.

Table 14: Clock Summary.

Clock	Waveform(ns)	Period(ns)	Frequency(MHz)
sys_clk_pin	{0.000 5.000}	10.000	100.000

Question 7

Générez le bitstream et vérifiez que vous avez le comportement attendu sur carte.

- Fichier de contraintes configurant les ports I/O du FPGA: `src/const/led_jukebox.xdc`
- Bitstream (2 MB): `impl/led_jukebox.bit`

Démonstration

Pour cette démonstration:

- Le *reset* est configuré pour une durée de 3 cycles d'horloge: `RST_SYN_STAGE = 3`
- Le diviseur de fréquence est configuré afin que, pour une horloge 100 MHz, un cycle allumé/éteint dure 2 secondes: `D = 2E8`

Scénario de démonstration vérifié sur carte Cora Z7 (`xc7z010c1g400-1`):

1. Au démarrage, la LED rouge clignote.
2. *Clicker* sur le bouton `BTN1` (presser puis relâcher): le circuit ne change pas de comportement.
3. *Clicker* sur le bouton `BTN0`: le code couleur bleu est enregistré dans la mémoire FIFO, puis lu lors de la prochaine transition allumé/éteint, et la LED bleu clignote. Elle clignotera jusqu'à ce qu'un autre code couleur soit disponible en sortie de la mémoire FIFO.
4. Presser et maintenir le `BTN1`, *clicker* trois fois sur le bouton `BTN0`, relâcher le bouton `BTN1` et *clicker* une dernière fois sur le bouton `BTN0`: la LED verte clignotera 3 fois, puis la LED bleue clignotera à nouveau (dernière couleur enregistrée).
5. Presser et maintenir le `BTN1`, puis *clicker* sur le bouton `BTN0` (et enfin relâcher le bouton `BTN1`): la LED verte clignote (unique couleur enregistrée).